Swiggy Business & Operations Analytics

A teaching guide to the complete project

Audience: beginner-to-intermediate data analysts
Project type: educational portfolio case study using synthetic data
Tools: Python, pandas, PostgreSQL SQL, Power Query, DAX, Power BI, Git
Source attribution: adapted from a public educational project

> The included records are synthetic. The results in this guide describe this dataset only; they are not claims about Swiggy's real business performance.



1. What this project teaches

This repository is an end-to-end example of taking transactional CSV files and turning them into analyzed, documented outputs. A learner can follow the work through these stages:

1. Understand the business questions and source data.
2. Check row counts, missing values, duplicates, keys, and data types.
3. Calculate and explain KPIs with Python and SQL.
4. Design a model for filtering business metrics by customer, restaurant, city, cuisine, and time.
5. Build and validate report pages and measures in Power BI.
6. Communicate observations, limitations, and recommendations honestly.
7. Track and share the work using GitHub.

The default demo uses CSV and Python; PostgreSQL is optional. Power BI Desktop is needed to edit the `.pbix` file. A public Power BI link is not included.

2. Repository map

The repository retains the original educational materials and places portfolio adaptations alongside them. The folders are not all interchangeable.

Path	What it contains	How to use it
`Data/raw/`	Five source CSV tables: users, restaurants, menu, orders, and order items	Treat as read-only inputs; use them for Python, SQL, and Power BI
`Data/processed/`	A README describing the intended location for model-ready data	The CSV pipeline writes generated summaries to lowercase `data/processed/`
`Database/`	Original PostgreSQL table, data-load, and aggregation scripts plus a portable loader	Optional PostgreSQL learning path; inspect paths and credentials before use
`Scripts/`	Original project scripts, including data generation and database helpers	Reference material; the CSV-first pipeline is under `python/scripts/`
`Notebooks/`	Original exploratory-analysis and data-analysis notebooks	Read and run interactively in Jupyter; outputs can be machine-specific
`Dashboard/`	Original Power BI report and original DAX notes	Retained reference assets; the portfolio report is under `powerbi/`
`Docs/`	Original project documentation and reports	Reference material, retained for attribution and context
`Visuals/`	Original project image and visualization assets	Reference images; not all are current report screenshots
`python/scripts/`	CSV pipeline, data-quality check, and EDA summary	Run the reproducible Python demonstrations
`python/notebooks/`	Notes about the portfolio notebook location	The original notebooks remain in `Notebooks/`
`sql/`	Portfolio schema, data-quality, and business-analysis SQL examples	PostgreSQL learning and query reference
`powerbi/`	Portfolio `.pbix`, screenshots, and report documentation	Open the report in Power BI Desktop and follow its refresh guide
`Docs/`	Portfolio requirements, data dictionary, model, insights, and this guide	Read these before presenting results
`README.md`	Short project introduction and setup entry point	Start here; link to this guide and PDF
`requirements.txt`	Python package requirements	Install in a virtual environment
`setup_windows.ps1`	Windows setup helper; PostgreSQL setup is optional	Review its local-only defaults before enabling the database path
`.gitignore`	Ignore rules for local environments, secrets, caches, and temporary data	Check before staging new files

Path	What it contains	How to use it
`.gitattributes`	Git LFS rules for `.pbix` and `.zip` binaries	Needed to handle large binary assets on GitHub
`LICENSE`	MIT license retained from the reference project	Preserve it and the original author attribution

Important distinction between portfolio and reference assets

The `python/`, `sql/`, `Docs/`, and `powerbi/` paths contain the portfolio additions. The capitalized `Data/`, `Database/`, `Docs/`, `Scripts/`, `Notebooks/`, `Dashboard/`, and `Visuals/` paths retain original/reference materials. When describing the project, identify which analysis or report you personally validated or changed. Do not imply that the synthetic source data or all historical assets were created by you.

3. The source data

The five CSV files are related but have different levels of detail. Identifiers should generally be imported as text, even if their values look numeric.

File / table	Rows	Grain (one row represents...)	Important fields
`users.csv` / `users`	10,000	One customer	`user_id`, `age`, `gender`, `marital_status`, `occupation`
`restaurants.csv` / `restaurants`	1,000	One restaurant	`restaurant_id`, `restaurant_name`, `city`, `cuisine`, `rating`, `is_cloud_kitchen`
`menu.csv` / `menu`	15,658	One menu item	`menu_id`, `restaurant_id`, `item_name`, `category`, `price`, `is_veg`
`orders.csv` / `orders`	600,000	One order	`order_id`, `user_id`, `restaurant_id`, `order_date`, `delivery_time`, `order_status`, `payment_method`, `total_amount`
`order_items.csv` / `order_items`	1,667,399	One item line within an order	`order_item_id`, `order_id`, `menu_id`, `quantity`, `price`

These data volumes and the example metrics below were validated against the included CSVs. Each source has no missing values or duplicate rows under the pipeline's basic whole-row checks. Those checks do not by themselves prove that all foreign keys match, values are valid, or the data is realistic.

Questions to ask about the data

- Is each identifier unique in the table where it is a primary key?
- Does every order refer to an existing customer and restaurant?
- Does each order-item line refer to an existing order and menu item?
- Are dates, times, prices, quantities, ratings, and status categories valid?
- Do `orders.total_amount` and summed order-item amounts reconcile?
- Are canceled orders included in the revenue definition?

The dataset documentation states that order status is limited to `Delivered` and `Cancelled`. It has order time-of-day but not pickup and drop-off timestamps, driver identifiers, route events, or actual delivery-duration observations.

4. Data model and grain

The logical relationships are:

```
users       1 ── * orders * ── 1 restaurants
                              |
                              1
                              |
                              *
                         order_items
                              *
                              |
                              1
                             menu
```

- `orders` is the order-level fact table for order count, amount, and status.
- `order_items` is a second fact table at the order-line grain.
- `users`, `restaurants`, and `menu` provide descriptive attributes.
- `restaurants` also describes the restaurant associated with each menu item.

The word **grain** means what one row represents. It is essential when joining tables. If an order has three item lines, joining its one `orders` row to those three `order_items` rows repeats its order-level `total_amount` three times. Do not sum order revenue after a one-to-many join unless the measure is designed to avoid that double counting. Use order-level totals from `orders`; use line quantity and menu item details from `order_items` and `menu`.

For Power BI, use one-to-many relationships from unique dimension keys to fact foreign keys, with single-direction filtering where practical. Avoid adding redundant paths that make a filter travel from one table to another in multiple ways. Confirm the actual `.pbix` model in Model view; this diagram is the documented target architecture, not a substitute for inspecting the report.

5. How the analysis runs

Recommended Windows CSV-first route

From the repository root in PowerShell:

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python .\python\scripts\csv_analytics_pipeline.py
python .\python\scripts\data_quality_checks.py
python .\python\scripts\eda_summary.py
```

Alternatively, the environment can be prepared with:

```
powershell -ExecutionPolicy Bypass -File .\setup_windows.ps1
```

The pipeline reads `Data/raw/` and creates three analysis outputs under `data/processed/`: `key_metrics.csv`, `city_revenue.csv`, and `customer_revenue.csv`. These are derived artifacts; rerun the script to re-create them from the raw inputs.

What each Python script demonstrates

`python/scripts/csv_analytics_pipeline.py`

1. Resolves the project root from the script location.
2. Confirms that the five required CSVs exist.
3. Loads each table with pandas and reports row counts.
4. Counts missing cells and duplicate whole rows.
5. Joins orders to restaurants to aggregate revenue by city.
6. Aggregates revenue by customer.
7. Writes the summaries and KPI row to `data/processed/`.

`python/scripts/data_quality_checks.py` reports each source's rows, columns, missing cells, duplicate rows, order-status counts, and numeric summary for order amounts.

`python/scripts/eda_summary.py` reports order and revenue summaries, city revenue, highest-revenue customers, and status mix. Its outputs are starting points for analysis, not a complete outlier, statistical, or causal study.

Optional PostgreSQL learning route

`sql/schema/01_create_schemas.sql` contains PostgreSQL DDL for the project tables. `sql/data_cleaning/01_data_quality_checks.sql` contains checks, and `sql/business_analysis/01_business_kpis.sql` contains example analytical queries. The original `Database/` scripts and portable loader are also retained.

PostgreSQL is not needed to run the default CSV pipeline. Before running any database loader, inspect the script and set up a local database using credentials that are not committed to Git. Do not reuse sample credentials on a shared or production server. Never stage generated local files such as `Database/Insert_Data_Local.sql` if they contain local paths or secrets.

Power BI route

Open `powerbi/Swiggy_Business_Analytics.pbix` in Power BI Desktop. The report pages observed in Power BI Service are:

1. Executive Overview
2. Customer Intelligence
3. Revenue Analytics
4. Restaurant Performance
5. Menu & Product Intelligence
6. Operations & Efficiency
7. Market Expansion

Use the screenshots in `powerbi/screenshots/` as previews. Follow `powerbi/documentation/powerbi_refresh_checklist.md` before refreshing or presenting. The Power BI Service report is not currently documented here as publicly accessible; a public URL must only be added after the tenant permits Publish to web and Power BI generates the link.

6. KPI definitions and interpretation

The CSV pipeline currently uses these definitions:

KPI	Definition in the pipeline	Interpretation caution
Total Orders	Number of rows in `orders`	Assumes one row per order; check unique `order_id`
Total Revenue	Sum of `orders.total_amount`	Includes every row, including canceled orders, unless source amounts or a later filter say otherwise
Average Order Value	Mean of `orders.total_amount` across all rows	Uses the same all-status population as the current revenue calculation
Total Customers	Distinct `orders.user_id` count	Counts customers who appear in orders, not necessarily all registered users
Cancellation Rate	Canceled-order rows divided by all order rows	This is a count rate, not cancellation value or cause
City Revenue	Sum of order amount after joining on `restaurant_id`, grouped by restaurant city	Validate unmatched restaurant IDs and revenue population

The current DAX notes are in `powerbi/documentation/dax_measures.md`. For example, a measure should use `DIVIDE` for safe division and a proper date table for time intelligence. Growth percentages require a clearly defined comparison period and a complete, related calendar table.

Reconciliation issue to resolve before presenting

The CSV pipeline was verified at **INR 416,956,363.49 total revenue** and **INR 694.93 average order value**. The Power BI Service report currently shows **INR 39.62 crore** total revenue and **INR 694.67 AOV**. These figures do not match exactly. A possible explanation is different status filters or measure definitions, but that has not been confirmed. Inspect filters, relationships, and DAX in Power BI, agree on whether canceled orders count as revenue, then reconcile the visuals against the CSV and SQL calculations before stating that the dashboard matches the pipeline.

The Power BI page plan describes delivery-time and delayed-order analysis as conditional. The source data has no actual trip-duration events, so do not claim average delivery time, delayed orders, or driver performance from these files.

7. Dataset-based observations

The following are verified summaries of the synthetic dataset:

- 600,000 order rows and 10,000 distinct customer IDs in orders.
- INR 416,956,363.49 summed order amount using the pipeline's all-row formula.

- INR 694.93 mean order amount using the same population.
- 4.95% canceled orders by count (29,719 of 600,000).
- Mumbai has the highest city revenue in the pipeline aggregation, about INR 92.16 million.
- Average order count per customer is 60, but the median is 13; customer frequency is not evenly distributed.

These observations are descriptive, not causal. For example, a high-revenue city does not prove that a particular marketing action caused the result. Recommendations in `Docs/business_insights.md` should be presented as hypotheses to test, not as proven operational interventions.

8. How to teach the project: guided lesson

Lesson A — inspect and frame the question

Ask learners to read `Docs/business_requirements.md` and choose one question, such as “Which cities contribute the most order value?” Have them specify the business definition, time range, status inclusion, and expected output before writing code.

Lesson B — profile the source

Run the data-quality script. Ask what its missing-value and duplicate checks can establish, and what they cannot. Have learners add key uniqueness and foreign-key checks. Compare the row count of `orders` with distinct `order_id`.

Lesson C — calculate a result twice

Calculate city revenue in Python and in SQL. Both analyses should join `orders.restaurant_id` to `restaurants.restaurant_id`, sum the chosen amount field, group by city, and sort descending. Check that totals reconcile under the same status and time filters.

Example SQL:

```
SELECT r.city,
       SUM(o.total_amount) AS revenue,
       COUNT(*) AS order_count
FROM orders AS o
JOIN restaurants AS r
  ON r.restaurant_id = o.restaurant_id
GROUP BY r.city
ORDER BY revenue DESC;
```

Lesson D — model and visualize

Sketch the table grain and keys before connecting Power BI. Create relationships, add a calendar table for time comparisons, and test each measure with a table visual before building cards and charts. Use slicers to verify that city, restaurant, customer, and date filters behave as intended.

Lesson E — communicate without overclaiming

For each insight, write four lines: observation, supporting metric, possible explanation, and action to test. State that the data is synthetic and mention the revenue discrepancy until it is resolved. Keep delivery-duration claims out of the narrative unless new event-level data is added.

Suggested learner exercises

1. Add a referential-integrity report for all foreign keys.
2. Define delivered-only revenue and compare it with all-order amount.
3. Add revenue share by city and verify shares sum to approximately 100%.
4. Create a calendar table and year-over-year measure with explicit period coverage.
5. Create customer frequency bands and document their cut points.
6. Explain why summing order amount after joining to line items can overstate revenue.
7. Write a one-page executive summary that separates facts from hypotheses.

9. Data limitations and responsible presentation

- The records are synthetic educational data, not live Swiggy customer or operational records.
- Dataset fields include customer attributes; retain only the minimum columns needed when creating derivative extracts.
- No actual driver IDs, pickup/drop timestamps, delivery duration, route logs, or delivery event history are present.
- The `orders` and `order_items` tables use different grains; careless joins can inflate measures.
- Growth measures need explicit dates, comparable periods, and a calendar table.
- The dashboard's displayed revenue does not currently reconcile to the raw CSV pipeline calculation.
- The Power BI Service report is not publicly accessible from the link state checked for this guide. GitHub can show images and host files, not run a `.pbix` interactively.
- `.pbix` and source archives use Git LFS; keep LFS enabled when cloning or downloading the full assets.

10. Attribution, license, and source history

This portfolio repository adapts and reorganizes a public educational reference. The MIT license is retained; review `LICENSE` for its terms. Identify your own contributions accurately and do not represent the source data, original visuals, or reference analysis as newly created work.

11. File-by-file learning index

File	Teaching purpose
`README.md`	Concise recruiter-facing overview, setup, and links
`requirements.txt`	Python package list
`setup_windows.ps1`	Windows environment bootstrap; optional PostgreSQL setup

File	Teaching purpose
`python/scripts/csv_analytics_pipeline.py`	Main reproducible CSV workflow and aggregations
`python/scripts/data_quality_checks.py`	Basic row, column, missing-value, duplicate, status checks
`python/scripts/eda_summary.py`	Summary-level exploratory analysis
`sql/schema/01_create_schemas.sql`	PostgreSQL table definitions, constraints, indexes
`sql/data_cleaning/01_data_quality_checks.sql`	SQL data-quality examples
`sql/business_analysis/01_business_kpis.sql`	Revenue, customer, restaurant, status, growth query examples
`Database/Create_Tables.sql`	Original PostgreSQL table setup
`Database/Insert_Data.sql`	Original data loader; inspect its machine-specific assumptions before use
`Database/Insert_Data_Portable.sql`	Portable-loader template used for optional local PostgreSQL setup
`Database/Create_Aggregated_table.sql`	Original aggregation-table reference
`Scripts/Swiggy_Data_Generator.py`	Original synthetic-data generation reference
`Scripts/Create_Tables.py`, `Scripts/Insert_Data.py`, `Scripts/Create_Aggregated_table.py`	Original database helper scripts
`Notebooks/Exploratory_Data_Analysis.ipynb`	Original EDA notebook
`Notebooks/Swiggy_Data_Analysis.ipynb`	Original analysis notebook
`Dashboard/DAX_Measures.md`	Original DAX reference
`Dashboard/Swiggy_Sales_Analysis.pbix`	Original Power BI reference report
`powerbi/Swiggy_Business_Analytics.pbix`	Portfolio Power BI report
`powerbi/documentation/dax_measures.md`	Portfolio DAX measure notes
`powerbi/documentation/report_pages.md`	Page plan and data support caveats
`powerbi/documentation/powerbi_refresh_checklist.md`	Refresh and validation steps
`Docs/business_requirements.md`	Business context, questions, and limits
`Docs/data_dictionary.md`	Field names and meanings
`Docs/data_model.md`	Relationship design and modeling guidance
`Docs/business_insights.md`	Dataset-based observations and recommendations
`powerbi/screenshots/`	Static previews for GitHub and teaching

File	Teaching purpose
`Docs/`	Original project problem statement, reports, and documentation
`Visuals/`	Original visualization assets retained as references
`.gitignore`	Excludes credentials, environments, caches, and local outputs
`.gitattributes`	Directs Power BI and ZIP binaries through Git LFS
`LICENSE`	Preserved license and attribution context

12. Glossary

Term	Meaning
AOV	Average order value under a stated order population
Cardinality	Relationship shape, such as one-to-many
Dimension	Descriptive table used to group or filter measures
Fact	Table of business events or measurable transactions
Foreign key	Column that points to a key in another table
Grain	What one row in a table represents
KPI	Key performance indicator with a defined formula and population
Referential integrity	Whether references point to valid records
Star schema	A model with fact table(s) connected to descriptive dimensions
Synthetic data	Artificially generated data used for development or education

13. PDF and source

This PDF is generated from `Docs/project_walkthrough.md`. To rebuild it on Windows, install the declared requirements and run:

```
python .\python\scripts\build_project_guide.py
```

The generator uses Microsoft Edge or Google Chrome in headless print-to-PDF mode. If neither browser is installed, the command reports the missing prerequisite. The editable Markdown file remains the source of truth.